

UNIVERSIDAD POLITÉCNICA --- DE SAN LUIS POTOSÍ

Act. 07: File path traversal simple case

Nivel: Apprentice

López Gómez Samuel-177393

CNO IV – Seguridad Informática

18/09/26

Servando López Contreras

Índice

Contenido

Índice	2
Tipo de Explotación	3
Objetivo del Laboratorio	3
Marco teórico y fundamentación técnica	3
¿Qué es Path Traversal?	3
Funcionamiento técnico	3
Variantes de la vulnerabilidad	4
Relación con el modelo CIA.....	4
Relación con OWASP Top 10.....	4
Explicación técnica de la vulnerabilidad	4
Procedimiento paso a paso.....	5
Explicación del Payload	15
Mitigación recomendada	15

Tipo de Explotación

Path Traversal / Directory Traversal (CWE-22) — vulnerabilidad de tipo *Broken Access Control*, clasificada dentro del **OWASP Top 10 (A01:2021 – Broken Access Control)**. Consiste en la manipulación de rutas de archivo suministradas por el usuario para acceder a recursos ubicados fuera del directorio restringido por la aplicación.

Objetivo del Laboratorio

Explotar la funcionalidad de carga de imágenes de la aplicación para leer el archivo `/etc/passwd` del servidor, sin contar con autorización ni credenciales, demostrando así un acceso no controlado al sistema de archivos subyacente.

Marco teórico y fundamentación técnica

¿Qué es Path Traversal?

Path Traversal es una vulnerabilidad que surge cuando una aplicación web utiliza datos proporcionados por el usuario para construir una ruta de acceso a un archivo o directorio, sin validar adecuadamente que dicha ruta permanezca dentro de los límites del directorio autorizado. El atacante inserta secuencias especiales (`../`) que le permiten "salir" del directorio base y navegar libremente por el sistema de archivos del servidor.

Funcionamiento técnico

En sistemas Unix/Linux, cada `../` representa un desplazamiento de un nivel hacia el directorio padre. Si una aplicación construye una ruta como:

```
/var/www/images/ + filename
```

y el atacante envía `filename = ../../../../etc/passwd`, el sistema operativo resuelve la ruta de la siguiente manera:

```
/var/www/images/../../../../../etc/passwd → /etc/passwd
```

El servidor, al no validar la entrada, interpreta la ruta resultante y entrega el archivo solicitado sin restricciones.

Variantes de la vulnerabilidad

- **Path Traversal simple:** uso directo de ../ sin ningún tipo de filtro (caso del presente laboratorio).
- **Path Traversal con filtros de secuencia:** la aplicación elimina ../ una sola vez, pero puede evadirse con secuencias anidadas como//.
- **Path Traversal con codificación:** uso de caracteres codificados en URL (%2e%2e%2f, %2e%2e/, ..%2f) para evadir filtros básicos de validación.
- **Path Traversal con rutas absolutas:** cuando la aplicación no valida rutas absolutas, permitiendo el acceso directo mediante /etc/passwd sin necesidad de retroceso.
- **Null byte injection (en sistemas legados):** truncamiento de la ruta mediante %00 para eliminar extensiones de archivo forzadas por el backend.

Relación con el modelo CIA

Esta vulnerabilidad impacta principalmente el pilar de **Confidencialidad** del triángulo CIA (Confidentiality, Integrity, Availability), ya que permite la divulgación de información sensible del sistema (credenciales, configuraciones, código fuente) sin autorización. En escenarios más avanzados, si el atacante logra además escribir archivos (Path Traversal de escritura), también se compromete la **Integridad** del sistema.

Relación con OWASP Top 10

Se clasifica dentro de **A01:2021 – Broken Access Control**, categoría que en la actualización de OWASP pasó a ser la más crítica, dado que agrupa vulnerabilidades donde los controles de acceso no restringen correctamente lo que un usuario puede ver o hacer. Path Traversal es uno de los CWE más representativos de esta categoría (CWE-22).

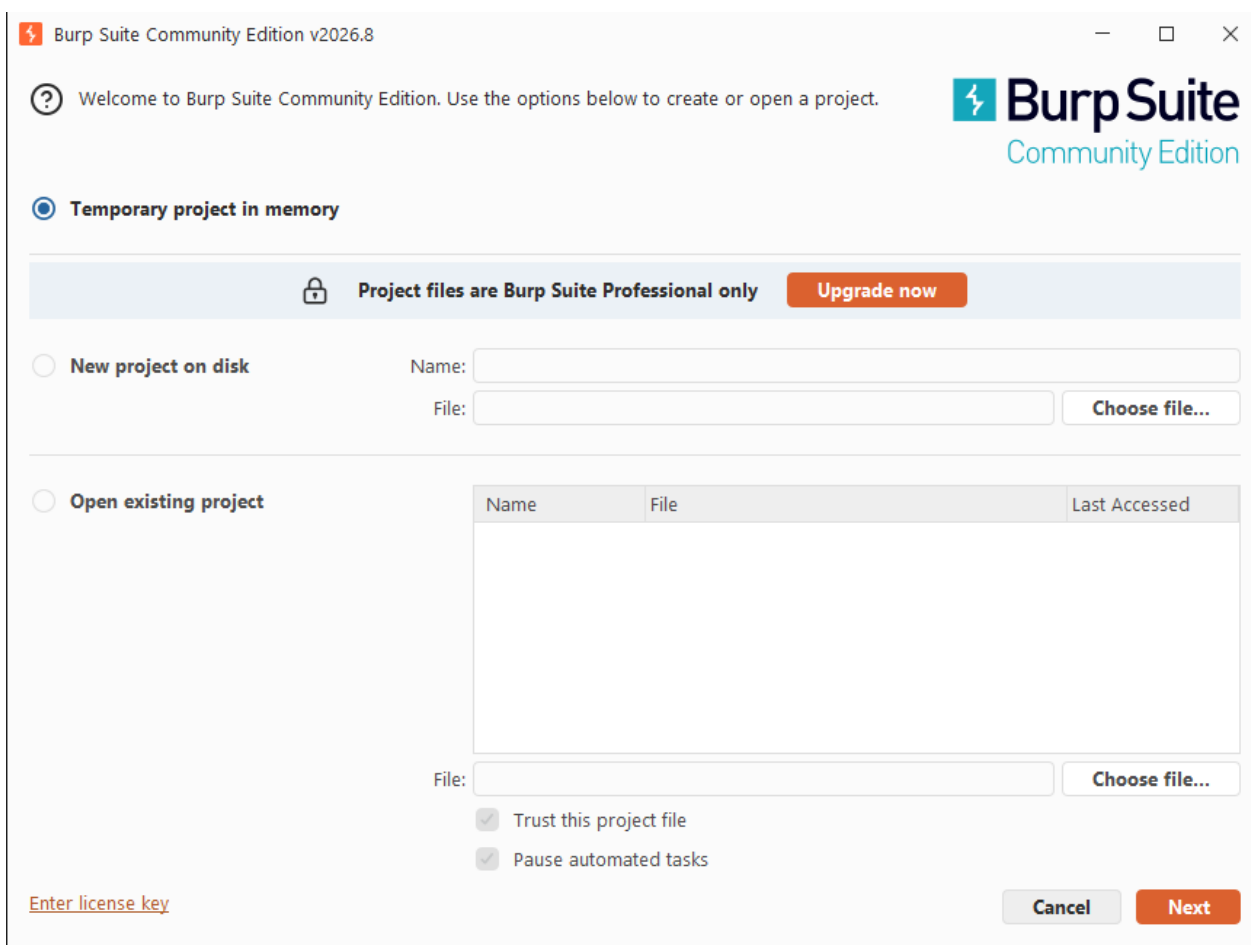
Explicación técnica de la vulnerabilidad

En este laboratorio específico, la aplicación implementa un endpoint que recibe un parámetro (por ejemplo filename) usado para servir imágenes de productos. El backend concatena este parámetro directamente a la ruta base de imágenes sin sanitización ni validación de tipo *whitelist*, ni comprobación de que la ruta resultante permanezca dentro del directorio esperado. Esto constituye un **caso simple de Path**

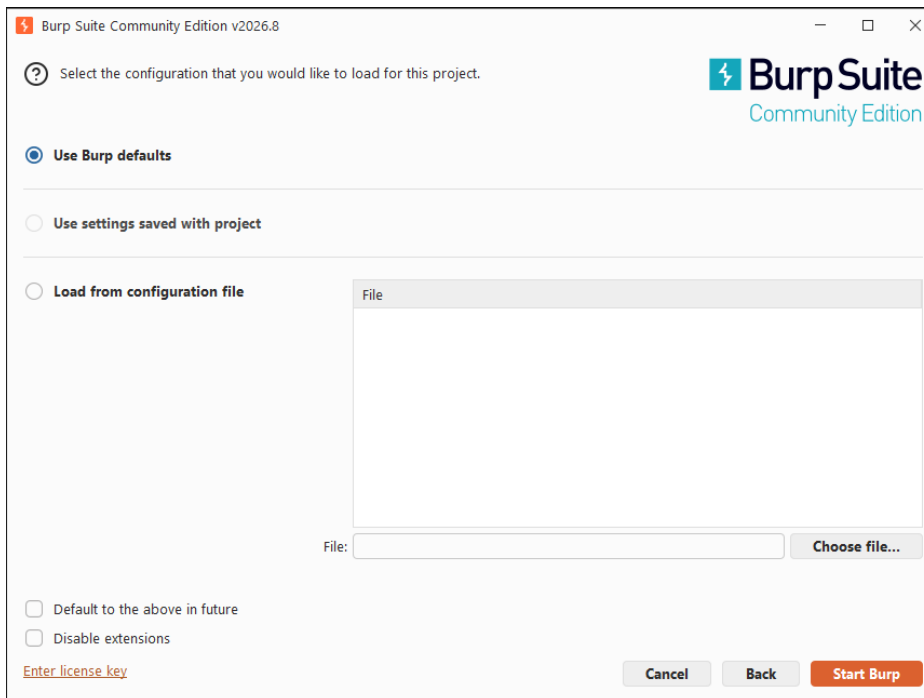
Traversal, sin ningún mecanismo de defensa implementado (sin filtros de secuencia, sin codificación requerida, sin validación de extensión), lo que lo hace directamente explotable con la secuencia estándar ../../../../etc/passwd.

Procedimiento paso a paso

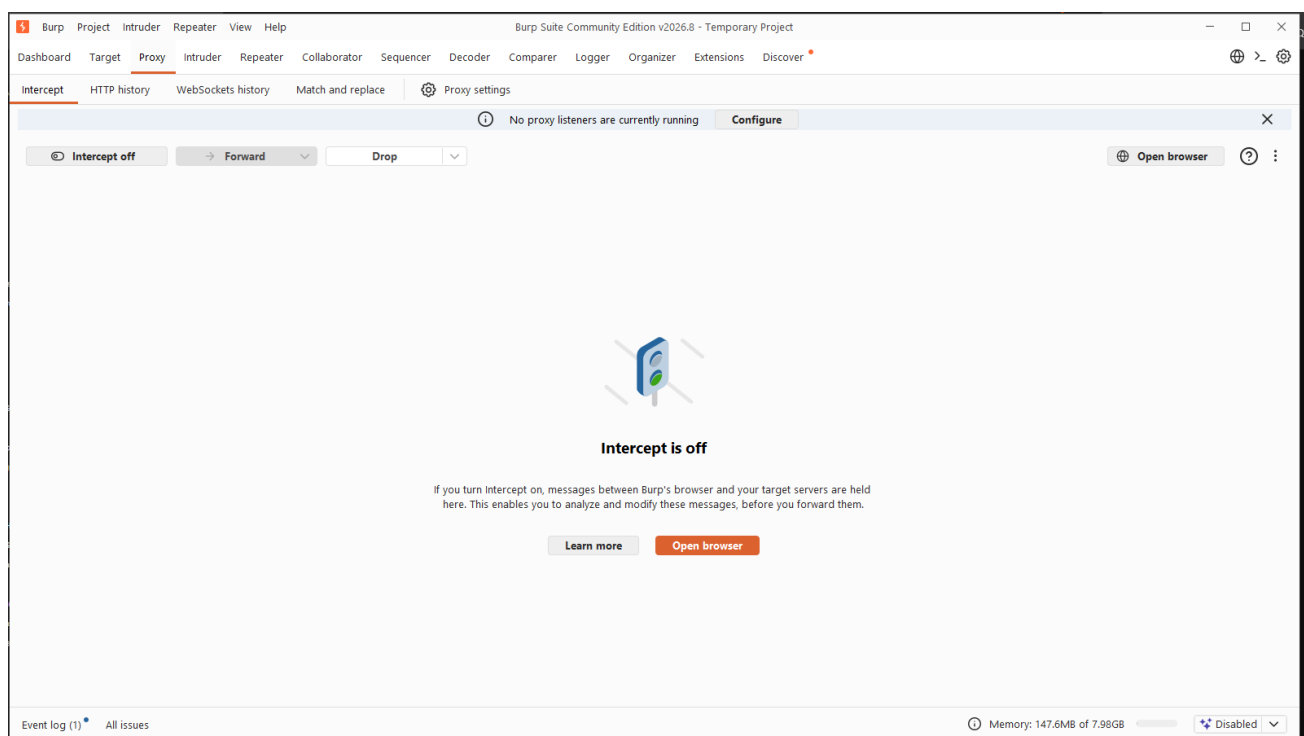
Se instaló y abrió Burp Suite Community Edition. En la ventana de bienvenida se seleccionó la opción Temporary project in memory, ya que no se requiere persistir el proyecto entre sesiones, y se dio clic en Next.



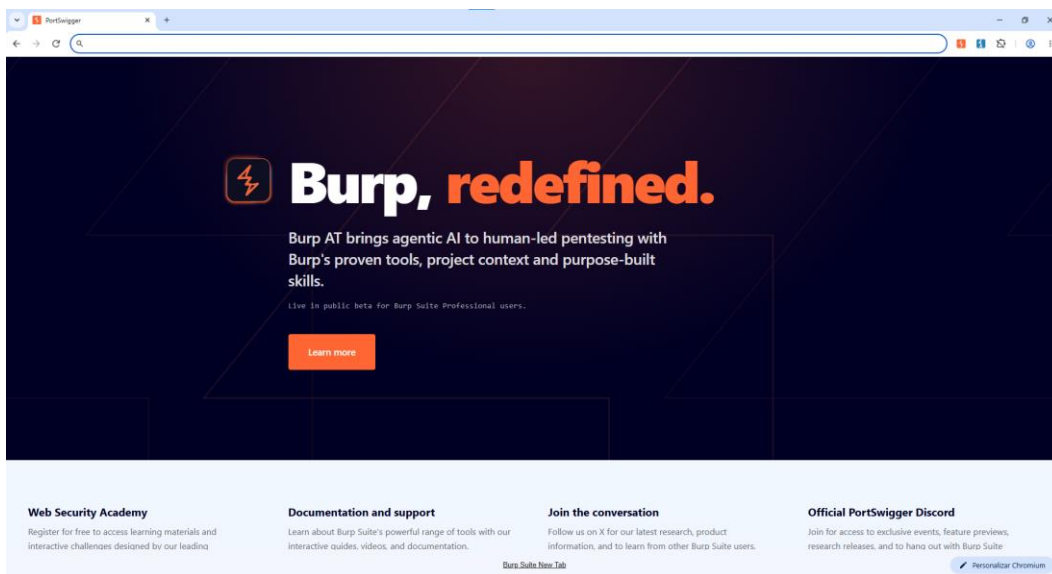
En la siguiente pantalla se seleccionó Use Burp defaults, configuración recomendada para un análisis estándar sin necesidad de perfiles personalizados, y se dio clic en Start Burp.



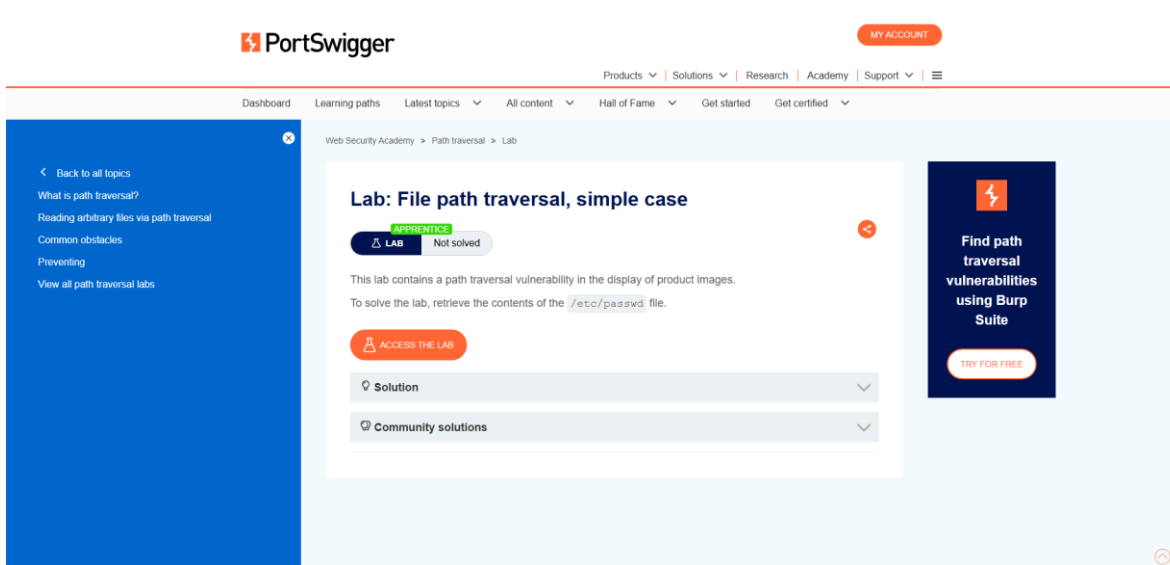
Se abrió la ventana principal de Burp Suite. En la pestaña Proxy > Intercept se observa que el interceptor se encuentra inicialmente desactivado (Intercept is off), estado desde el cual se procede a abrir el navegador integrado mediante el botón Open browser, ubicado en la esquina superior derecha del panel.



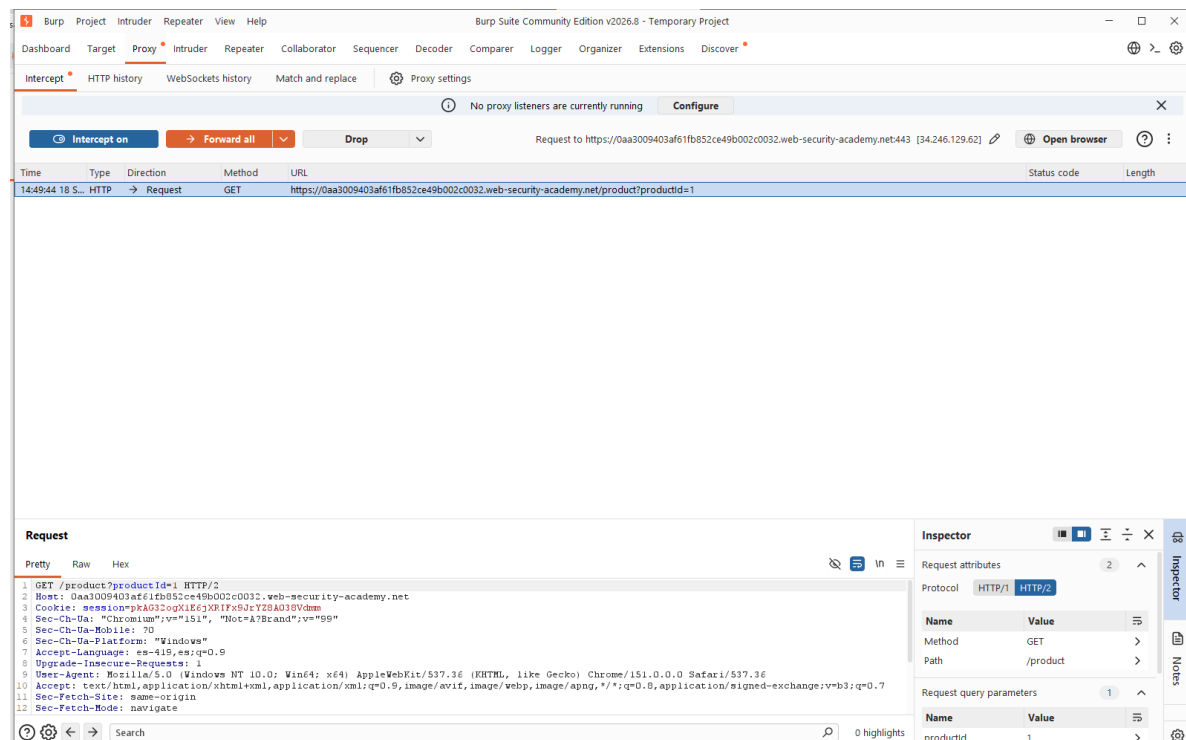
Al presionar Open browser se abrió una nueva ventana de navegador (Chromium) preconfigurada para enrutar todo su tráfico HTTP/HTTPS a través del proxy de Burp Suite, lo cual permite interceptar y analizar cada petición que se realice desde ese navegador.



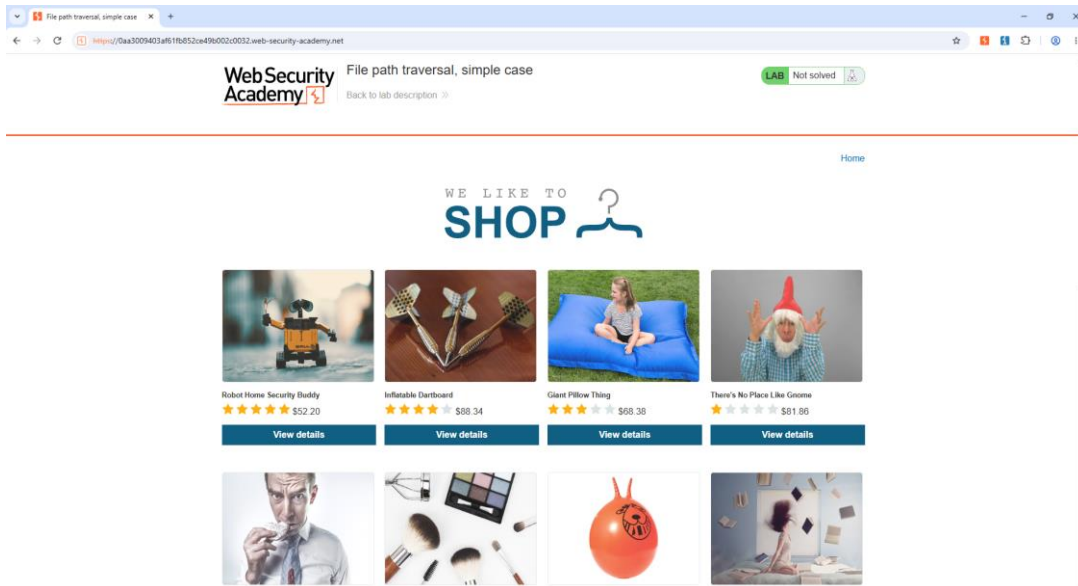
Desde el sitio oficial de PortSwigger se accedió al laboratorio "File path traversal, simple case" y se seleccionó el botón Access the lab, el cual genera una instancia única del laboratorio con una URL temporal asociada a la sesión de nuestra cuenta.



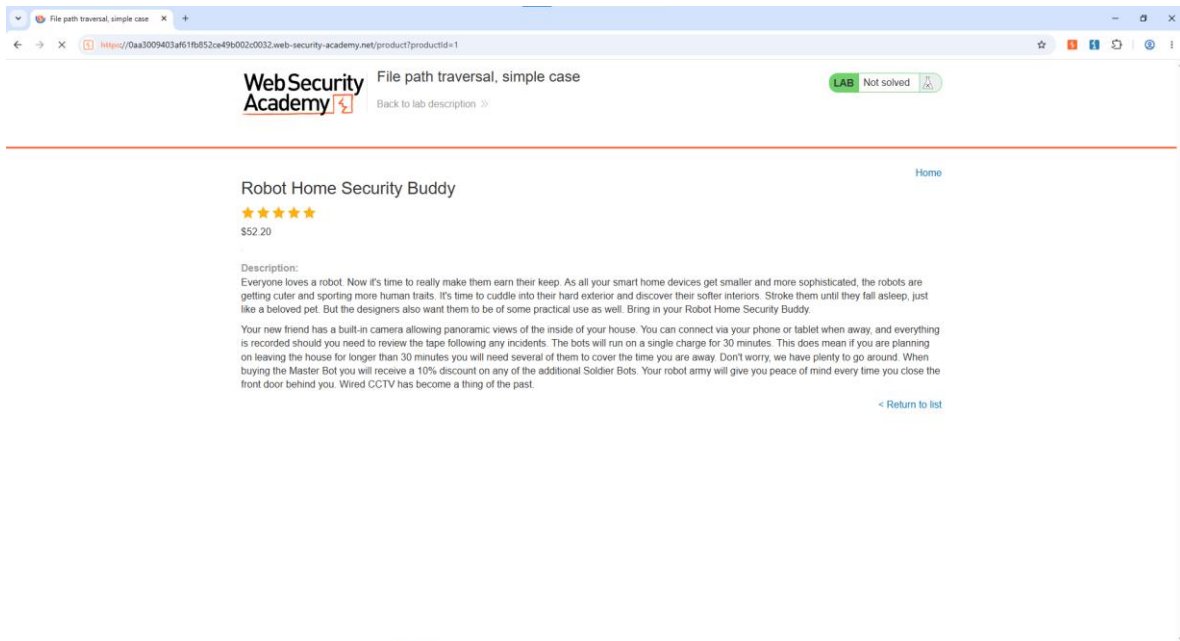
La URL generada se pegó en el navegador de Burp. Para esto es importante que tengamos la opción del interceptor activo, se registró en el histórico la primera petición GET realizada hacia /product?productId=1, visible en el panel Proxy, junto con sus encabezados (Host, Cookie de sesión, User-Agent, etc.) mostrados en el inspector de la derecha. Para que la pagina cargue correctamente es importante que seleccionemos la petición y demos click en el botón de Forward All



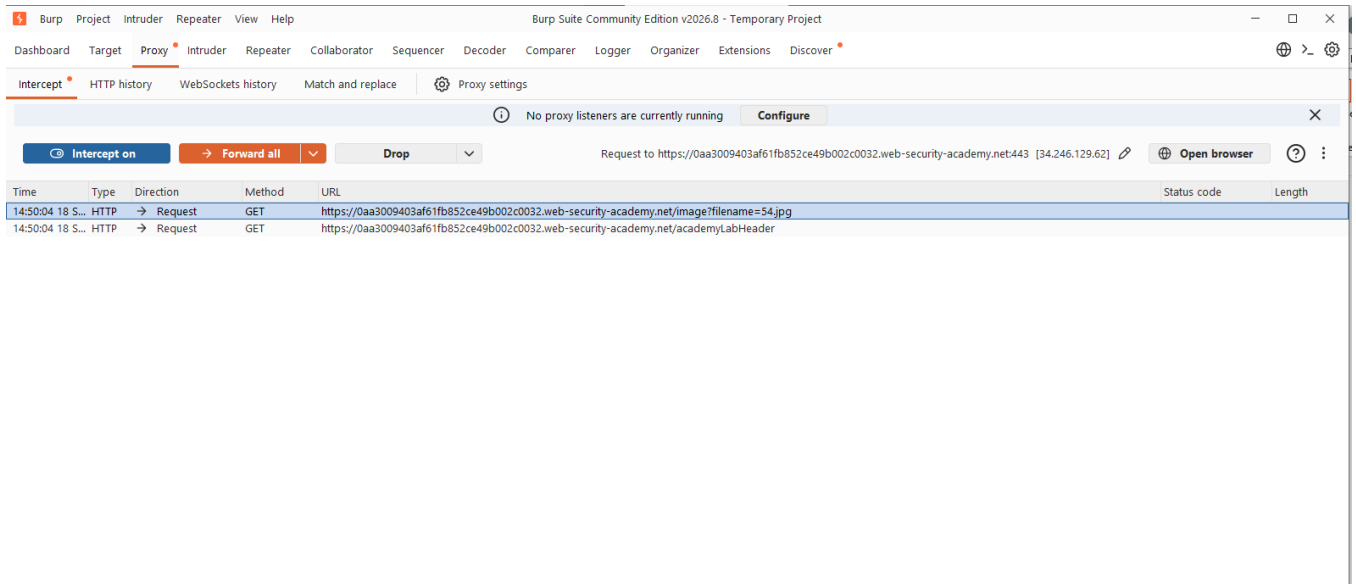
Al continuar la navegación, la página del laboratorio cargó correctamente, mostrando el catálogo de productos ("We Like To Shop") con sus respectivas imágenes en miniatura, cada una cargada mediante peticiones independientes al servidor.



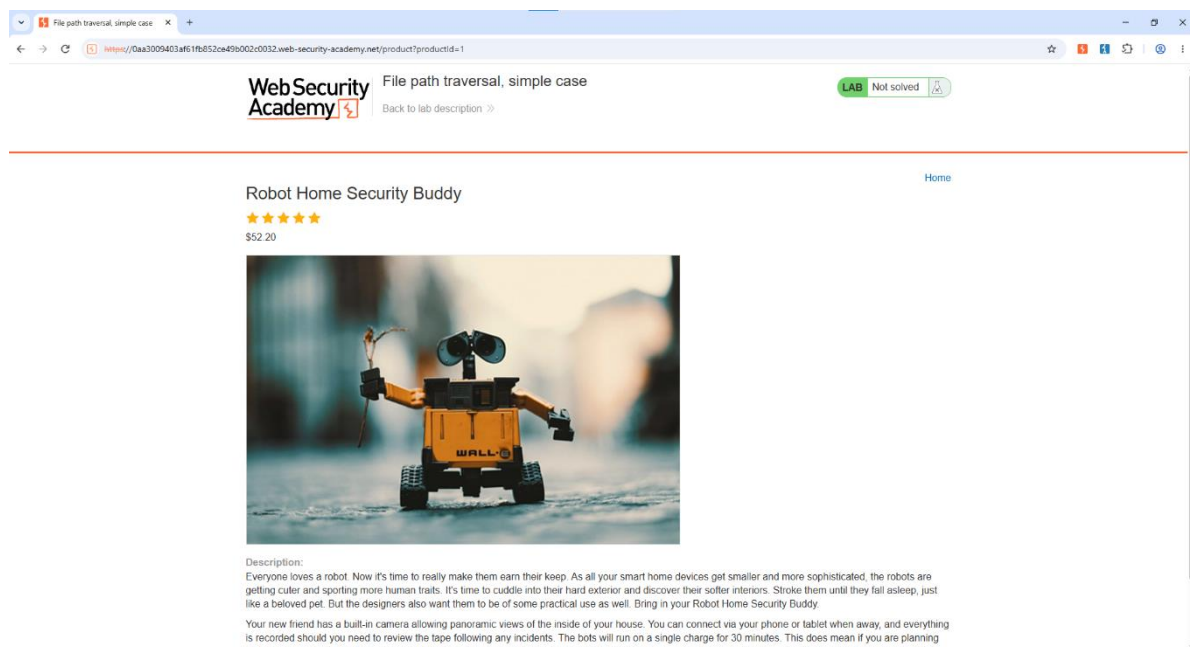
Se navegó hacia el detalle de uno de los productos ("Robot Home Security Buddy") mediante la opción View details, observando la ficha completa del producto, incluyendo descripción, precio e imagen ampliada. La página se muestra incompleta ya que tenemos que aceptar el trafico en el panel de burp.



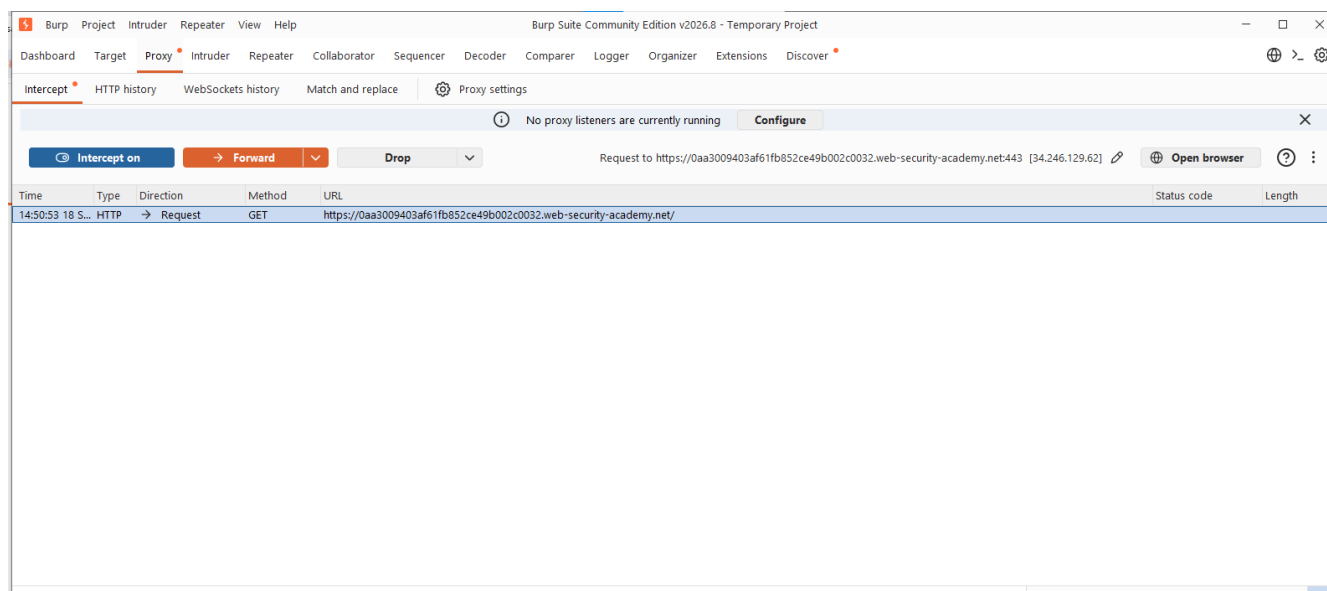
Regresando al panel de Burp, se observó que el tráfico correspondiente a la carga de imágenes quedó retenido en el proxy, evidenciando una petición GET hacia /image?filename=54.jpg y otra hacia /academyLabHeader, ambas pendientes de ser reenviadas.



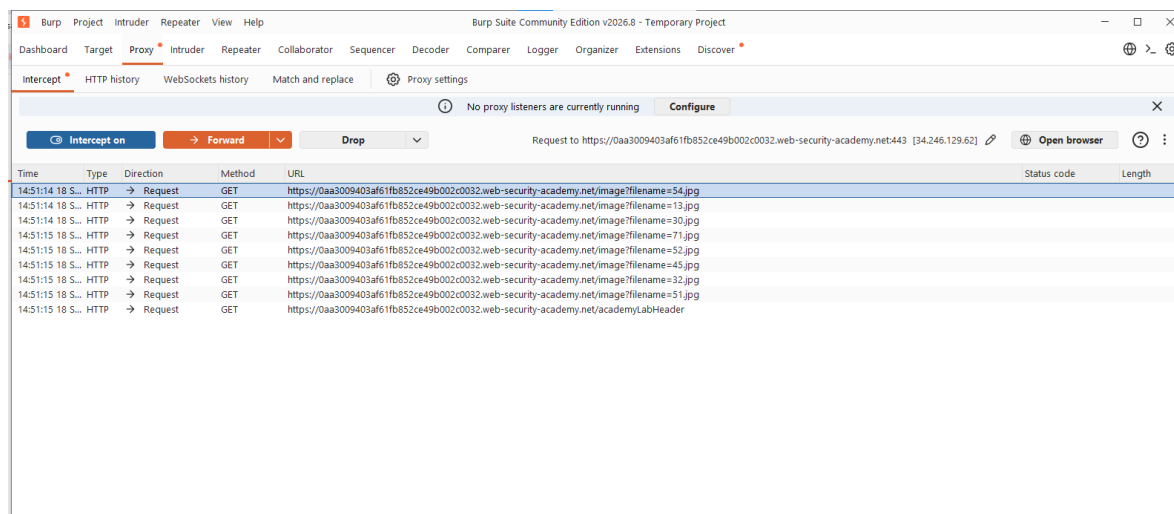
Al forzar (Forward) las peticiones interceptadas, la página terminó de cargar mostrando correctamente la imagen del producto, confirmando que el flujo normal de la aplicación depende de este endpoint /image?filename= para renderizar cada imagen.



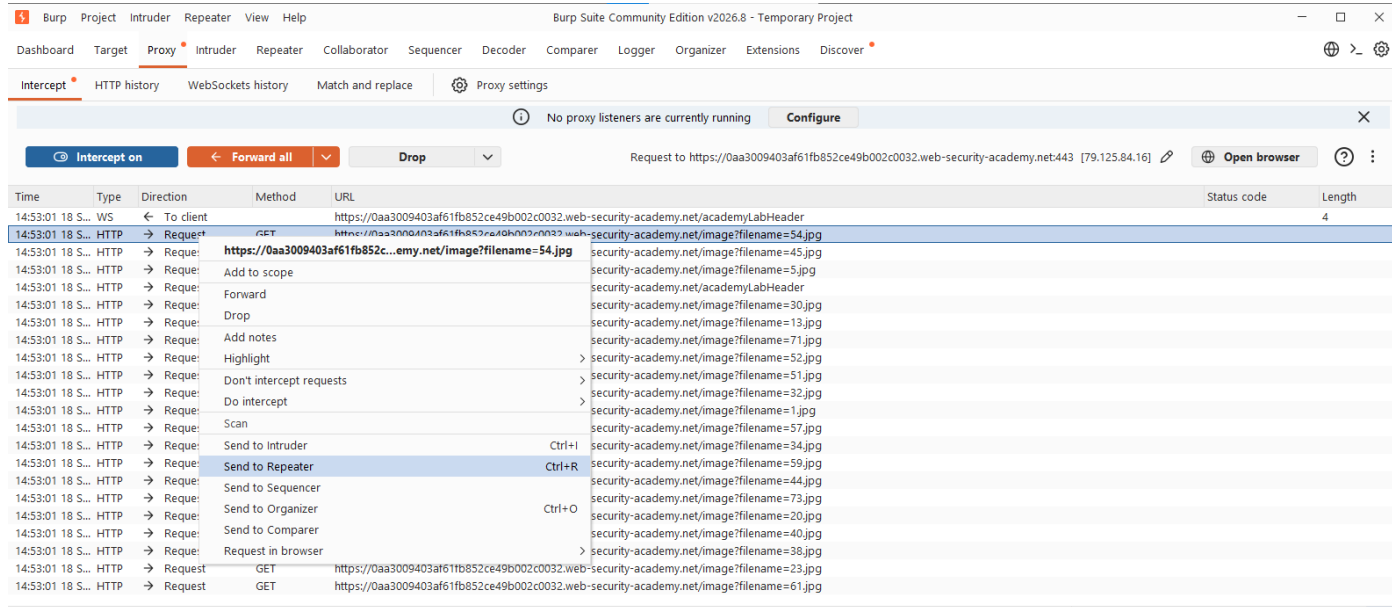
Con Intercept on activo nuevamente, dentro de la página seleccionamos el link Home esto para recargar la página principal del catálogo. En esta ocasión se identificó una única petición pendiente correspondiente a la carga inicial de la página (GET /).



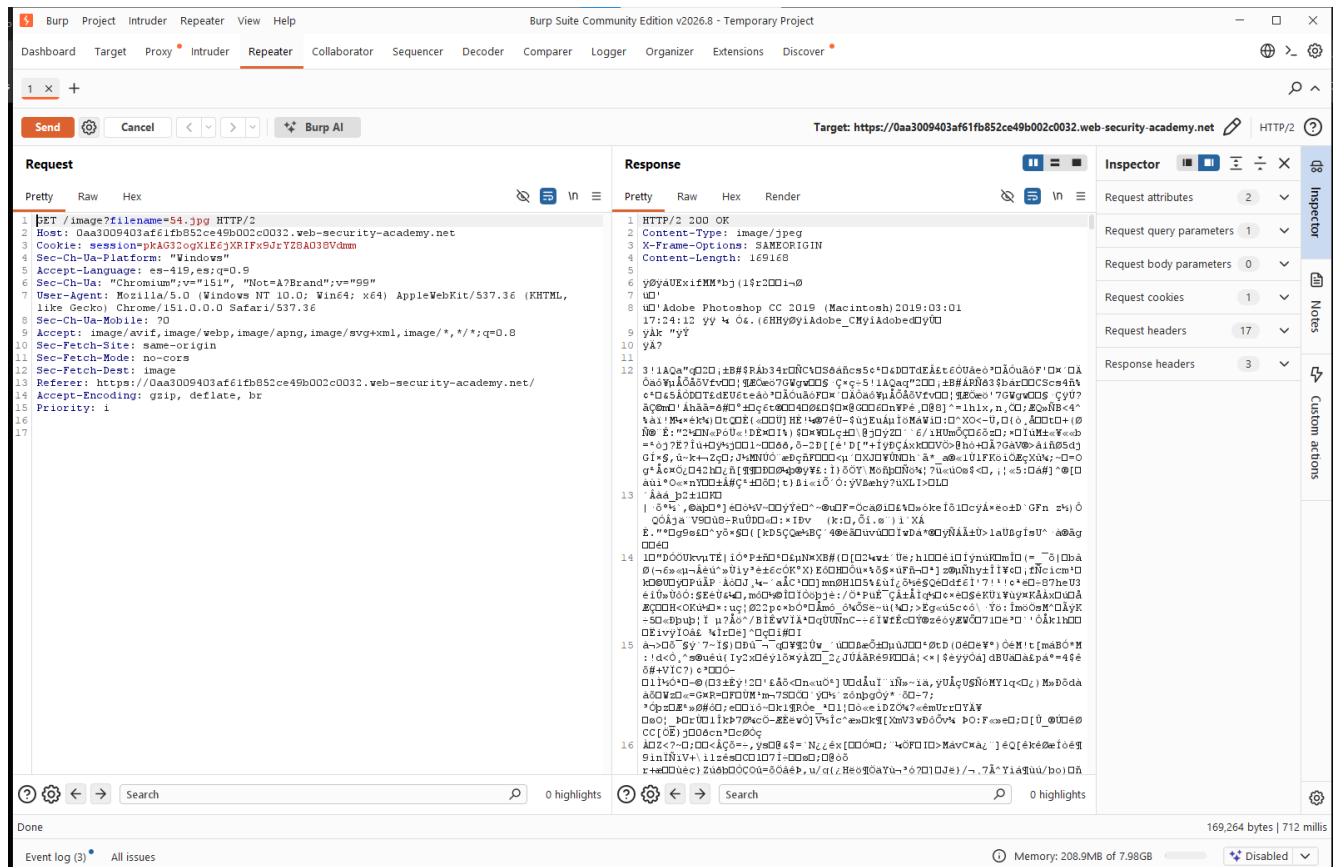
Al reenviar el tráfico nuevamente y recibir la petición de carga de la página principal se observó en el Proxy el patrón repetitivo de peticiones GET hacia /image?filename=NN.jpg, una por cada producto mostrado en el catálogo (filename=54.jpg, 13.jpg, 30.jpg, 71.jpg, 52.jpg, 45.jpg, 32.jpg, 51.jpg, entre otras). Este patrón común confirmó que el parámetro filename es el punto de entrada controlado por el usuario que la aplicación utiliza para construir la ruta del archivo de imagen a mostrar, siendo por tanto el vector idóneo para probar la vulnerabilidad de Path Traversal.



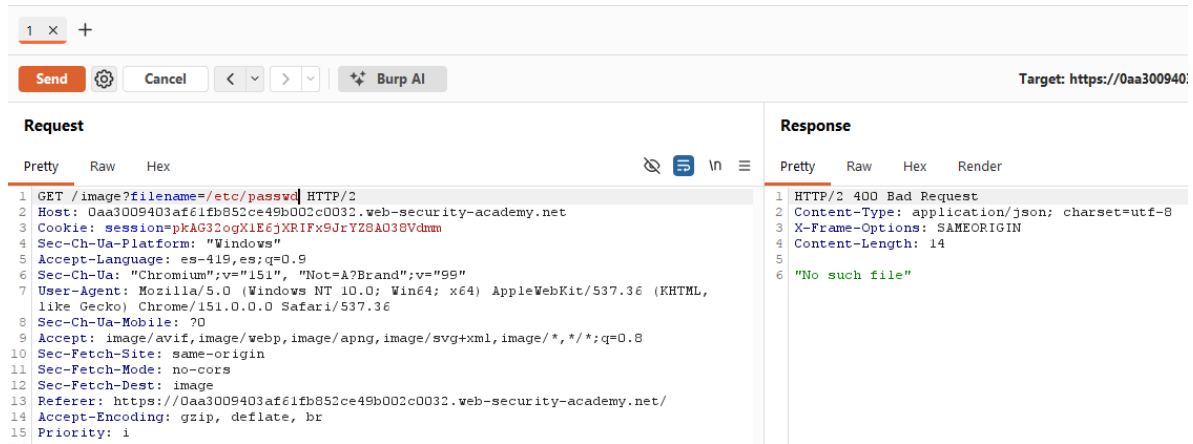
Se seleccionó una de las peticiones de imagen (filename=54.jpg) y, mediante clic derecho, se eligió la opción Send to Repeater, herramienta de Burp Suite que permite editar manualmente una petición HTTP y reenviarla las veces que sea necesario, observando la respuesta del servidor en tiempo real sin afectar el resto del flujo de la aplicación.



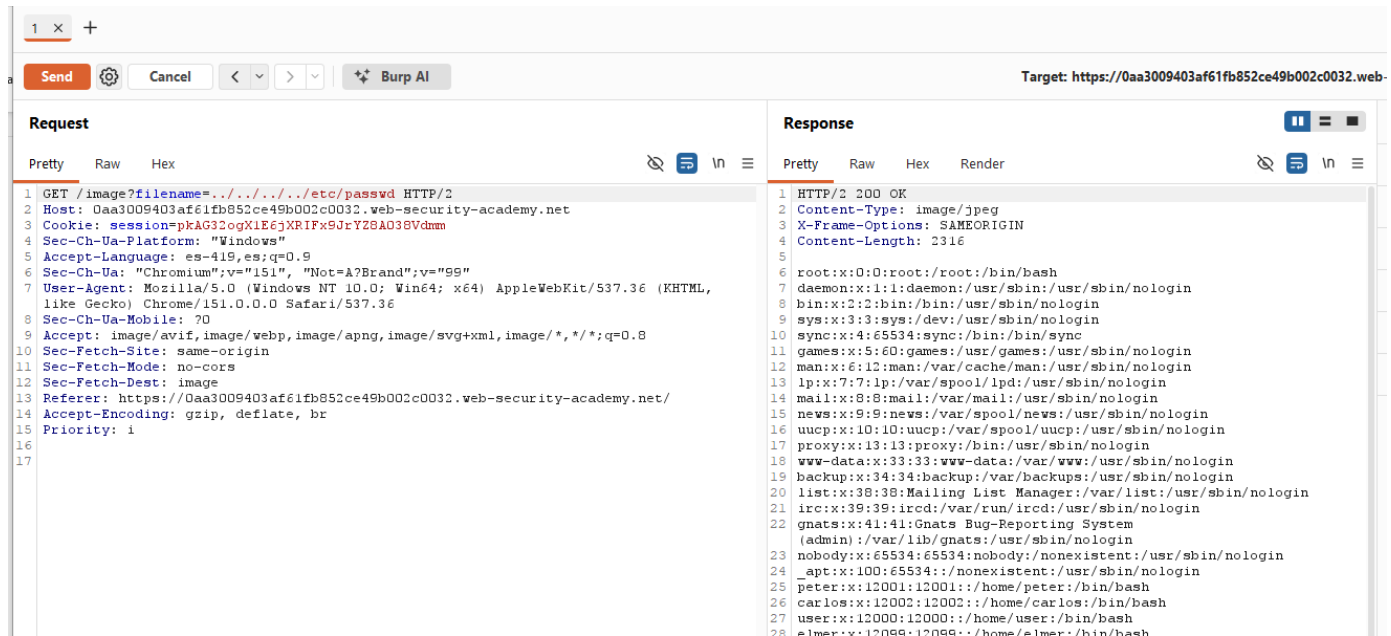
Ya en la pestaña Repeater, se confirmó el comportamiento normal de la aplicación enviando la petición original sin modificaciones mediante el botón Send. La respuesta del servidor mostró un código 200 OK con encabezado Content-Type: image/jpeg, devolviendo en el cuerpo de la respuesta el contenido binario de la imagen solicitada.



Como primer intento de explotación, se modificó el valor del parámetro filename a /etc/passwd (ruta absoluta simple, sin secuencias de retroceso) y se envió la petición. El servidor respondió con un código 400 Bad Request y el mensaje "No such file", indicando que la ruta no pudo resolverse desde el directorio base configurado.



Se modificó nuevamente el parámetro, esta vez utilizando la secuencia completa de traversal ../../../../etc/passwd, y se envió la petición mediante Send. En esta ocasión el servidor respondió con un código 200 OK y devolvió en el cuerpo de la respuesta el contenido íntegro del archivo /etc/passwd del sistema, listando las cuentas de usuario del servidor (root, daemon, bin, sys, www-data, peter, carlos, entre otras).



Al regresar a la pestaña del laboratorio en el navegador y actualizar la página, se confirmó que el estado del laboratorio cambió de "Not solved" a Solved, junto con el mensaje "Congratulations, you solved the lab!", validando la explotación exitosa de la vulnerabilidad.

 File path traversal, simple case
[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

Lab: File path traversal, simple case

APPRENTICE

LAB Solved

This lab contains a path traversal vulnerability in the display of product images.

To solve the lab, retrieve the contents of the `/etc/passwd` file.

Finalmente, se verificó en el panel de progreso del perfil de PortSwigger (Dashboard) el incremento del contador de laboratorios resueltos dentro de la categoría Apprentice, reflejando el avance registrado tras completar el ejercicio.

portswigger.net/web-security/dashboard

Not sure where to start? Structure your learning with one of our guided paths. [GET STARTED](#)

Your level

NEWBIE

Ne

Solve 60 more labs to become an apprentice.

Level progress

1 of 61 Apprentice	0 of 173 Practitioner	0 of 39 Expert
-----------------------	--------------------------	-------------------

See where you rank

- [Check out our Hall of Fame](#)

Hall of Fame high flyers

- [Read three of our user journeys](#)

Find your next topic

- [View all topics](#)

Your certifications

NOT READY

Burp Suite Certified PRACTITIONER

You're not ready to take the Burp Suite certification exam.

Vulnerability labs

[VIEW ALL](#)

0%

Exam preparation steps

NOT READY

The labs you choose to complete must be "Practitioner" level or higher.

0 of 23 Lab from all topics Read more	0 of 8 Specific labs Read more	0 of 5 Mystery labs Read more	0 of 1 Practice exam Read more
---	--	---	--

Explicación del Payload

```
../../../../etc/passwd
```

Desglose técnico del payload utilizado:

Componente	Función
../ (repetido 4 veces)	Retrocede cuatro niveles en la jerarquía de directorios desde la ubicación del recurso de imágenes hasta alcanzar la raíz del sistema de archivos (/).
etc/passwd	Ruta relativa, desde la raíz, hacia el archivo objetivo; en sistemas Unix/Linux este archivo almacena la lista de cuentas de usuario registradas en el sistema.

El uso de cuatro secuencias ../ asegura que, independientemente de cuán anidado se encuentre el directorio original de imágenes en el servidor (por ejemplo, algo similar a /var/www/app/images/), se alcance la raíz / del sistema de archivos. Una vez posicionado en la raíz, la ruta etc/passwd se interpreta de forma absoluta, permitiendo acceder al archivo sin necesidad de conocer previamente la estructura interna completa del servidor.

Es importante señalar que enviar más secuencias ../ de las estrictamente necesarias no genera error: el sistema operativo simplemente ignora los intentos de retroceso adicionales una vez alcanzada la raíz del sistema de archivos, lo cual explica por qué este tipo de payload suele probarse con varias repeticiones (3, 4, 5, 6...) cuando no se conoce la profundidad exacta del directorio base, como práctica habitual de fuerza bruta ligera durante la fase de explotación.

Por otro lado, el intento previo con el valor /etc/passwd (sin secuencias de retroceso) falló con un código 400, ya que el servidor lo interpretó como un nombre de archivo literal dentro del propio directorio de imágenes, es decir, buscó un archivo llamado literalmente "etc/passwd" dentro de /var/www/images/, el cual evidentemente no existe.

Mitigación recomendada

- Validación estricta de entrada (input sanitization): rechazar cualquier valor del parámetro que contenga secuencias como ../, ..\, o sus equivalentes codificados en URL (%2e%2e%2f, ..%c0%af).
- Uso de listas blancas (whitelisting): permitir únicamente nombres de archivo predefinidos o que coincidan con un patrón estricto (solo caracteres alfanuméricos y extensión permitida), evitando depender de listas negras fácilmente evadibles mediante codificación o anidamiento.

- Normalización y verificación de rutas (canonicalization): resolver la ruta absoluta final del archivo (por ejemplo, mediante funciones como `realpath()`) y verificar de forma explícita que dicha ruta permanezca dentro del directorio base autorizado antes de servir el contenido al cliente.
- Uso de identificadores indirectos: en lugar de recibir el nombre del archivo directamente desde el cliente, emplear un identificador (ID numérico o hash) que se mapee internamente, del lado del servidor, a la ruta física real del archivo correspondiente.
- Principio de menor privilegio: configurar los permisos del proceso del servidor de aplicaciones de modo que no tenga acceso de lectura a archivos sensibles del sistema operativo, tales como `/etc/passwd`, `/etc/shadow` o archivos de configuración internos.
- Defensa en profundidad: combinar la validación a nivel de aplicación con controles adicionales a nivel de sistema operativo (permisos restrictivos, entornos aislados tipo chroot jail o contenedores) y mecanismos de monitoreo y alertas ante patrones anómalos de acceso a archivos.